pw-12th nov-2

—

test function

to write tests.

syntax-

test(’test name’, callback function){

……}

test(’testname’, async({page})⇒{

console…….

}

page is called as fixture.

page denotes webpages.

—

test.only

run only this test case.

test.only(’testname, callbackfunction){

……

}

—

skip only this test.

test.skip(’testname, callbackfunction){

……

}

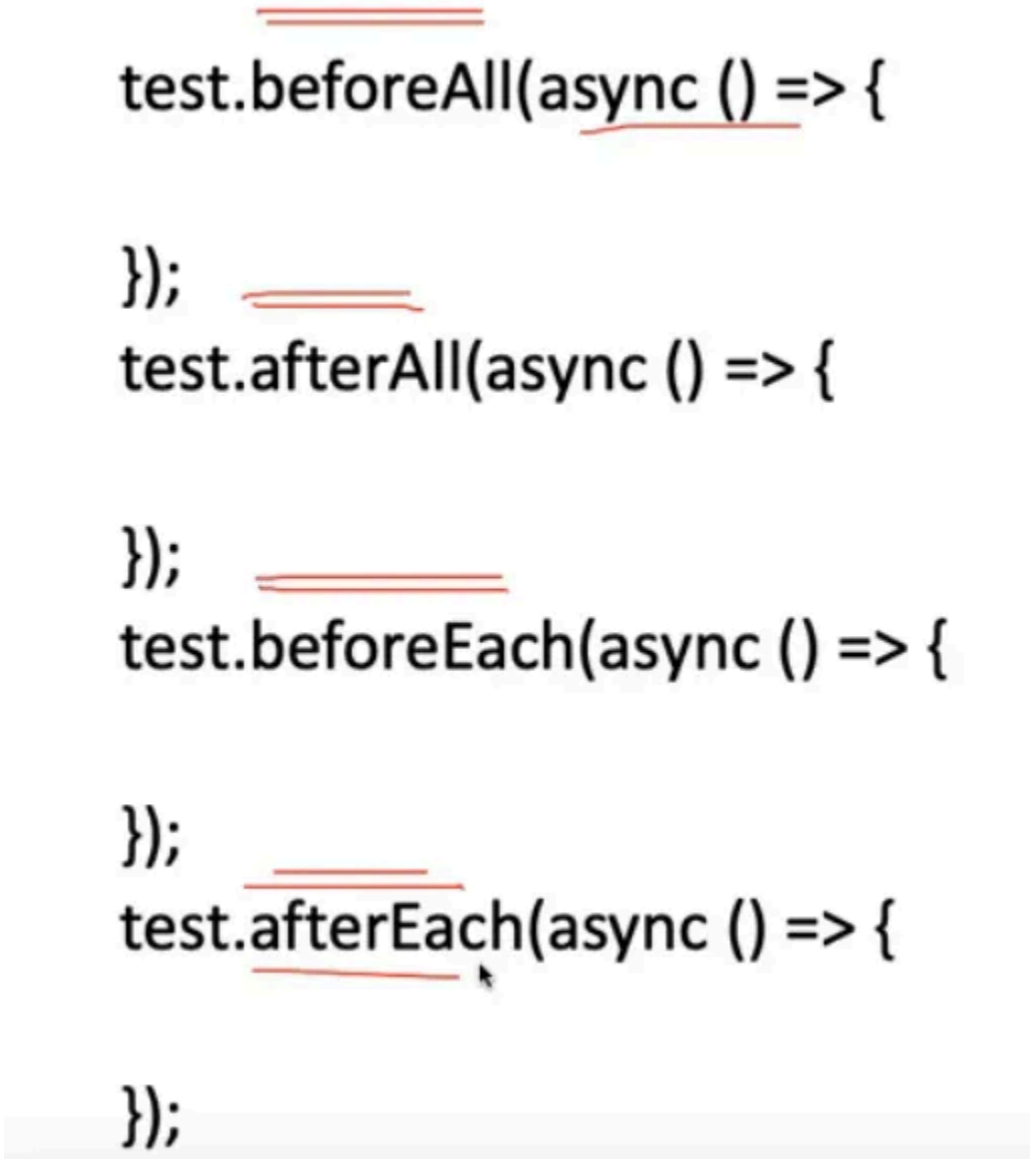
—more annotations-

before all the test.

after all the test.

before each test, after each test.

test name is not mandatory.



—

use method.

we can set anything at test level or at config level.

viewport is an example.

```
test.use({ viewport: { width: 100, height: 100 } });
```

—page fixture-

get fresh page instance for every test.

—request fixture-

for api testing.

```
test('basic test', async ({ request }) => {

});
```

—

browser fixture-

shared across all the pages opened in the same browser in new tabs.

```
test("window handle",async function ({ browser }){

})
```

-browsername fixture-

returns name of browser running the test case.

default when not specified is chromium.

```
test('skip this test in Firefox', async ({ page, browserName }) => {
  test.skip(browserName === 'firefox', 'Still working on it');

});
```

—plugin called **playwright test for vs code** download it.

—

incognito and default browser is chrome when not specified.

—to run the test using cli-

npx playwright test

—

describe block

write group of test.

no need of sync as its only grouping. even if we write no error.

async to write where we perform some actions.

—

expect for assertions.

comes from jest.

Expect

Expect gives you access to a number of "matchers" that let you validate different things.

expect(12).toBe(12)

expect(2.0).toBe(2.0)

expect("Mukesh Otwani".includes("Otwani")).toBeTruthy()

expect(true).toBeTruthy()

https://jestjs.io/docs/expect

—

protocol error - giving https or http is mandatory.

-url()

—title

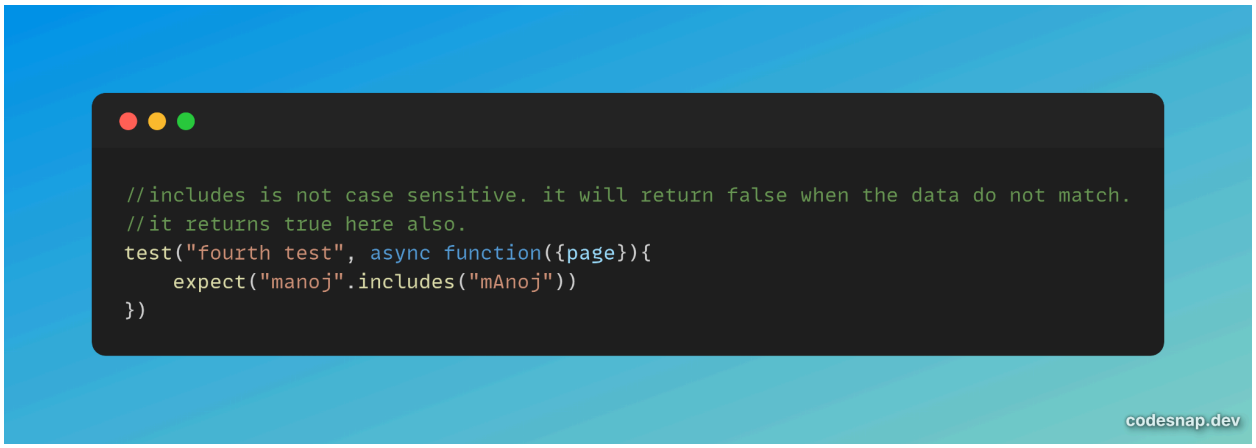
let titlevalue=await page.title();

—goto()

visit the url.

await page.goto("https://www.google.com")

—includes



—to be truthy and to be falsy-

—file name convention is important.

<filename>.spec.js

if not in this format then we wont be able to run.

—

better to have await with expect else error due to sync issues.

—

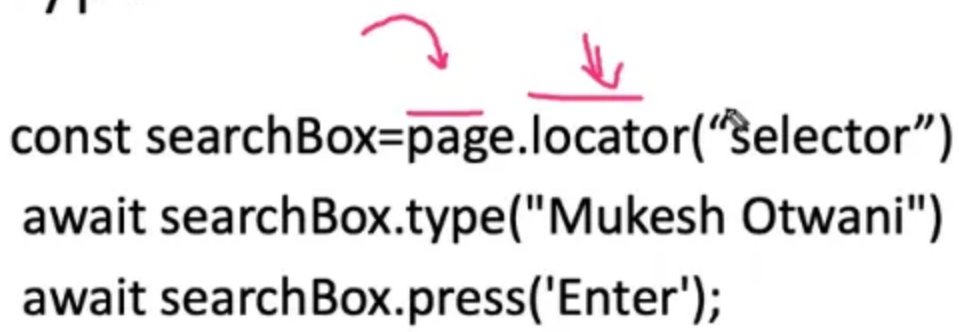
to run individual files using cli

npx playwright test "tests\\fourth.spec.js”

—locators-

we can pass xpath or css, no need to mention. returns webelement.

Type



press is for keyboard actions.

fill and type are same. in type keyboard actions allowed but not in fill.

locators will fail if more than one element matches. they are strict.

if more than one element use the below methods -

nth index - we pass index of element.

count() - to know how many elements are present of same locator.

First()
Last()
Nth(index)
Count()

-pw mandatorily checks these states and auto waits till moving to next step-

to ignore below checks use - forceful action using **force:true**

- element is Attached to the DOM
- element is Visible
- element is Stable, as in not animating or completed animation
- element Receives Events, as in not obscured by other elements
- element is Enabled
- element should be in viewPort

—go back method()-

—new methods of page.get

await page.getAttribute(<selector></selector>)

await page.getByAltText("alt text of image").click()

await page.getByLabel("get value from label attribute")

await page.getByPlaceholder("get value from placeholder attribute")

await expect(page.getByRole('heading', { name: 'Sign up' })).toBeVisible();

await page.getByTestId("id of webelement")

await page.getByText("visible text")

await page.getByTitle("title of page")

—we can store locators in variables and use them

delay will take that much seconds between words.

use locator for xpath or css.

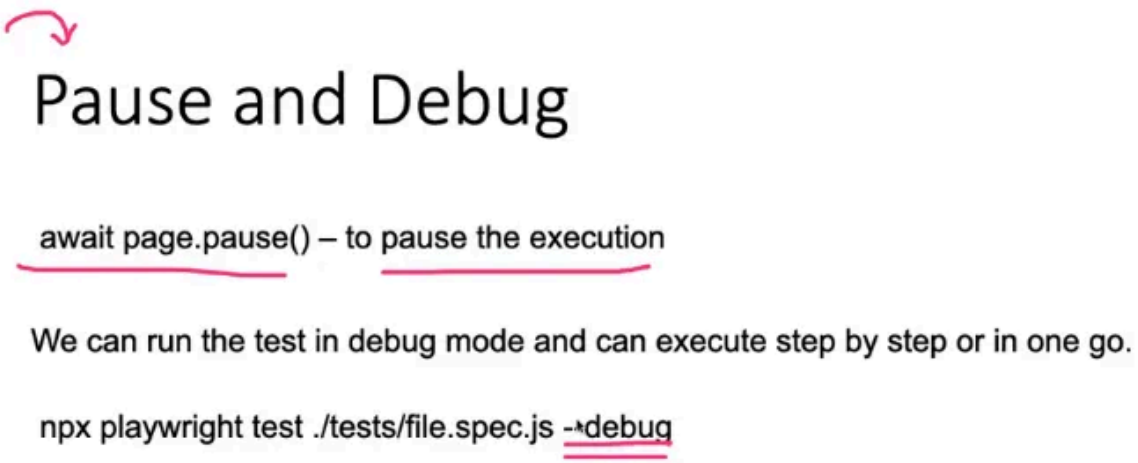
—click

for clicking.

await page.getByLabel("Testing").click()

—we get strict mode violation when we get multiple web elements for same locator when running the code.

—first method



—go back method()-
//go back by clicking on back button of browser.

```
   test("login 2",async function({page}){
    await page.goto("https://ineuron-courses.vercel.app/login")
    await page.goto("https://www.google.com")
    await page.goto("https://www.yahoo.com")

    await page.goBack()
    await page.goBack()
    await page.goBack()
   })
```

—new methods of page.get

```
await page.getAttribute(<selector></selector>)

await page.getByAltText("alt text of image").click()

await page.getByLabel("get value from label attribute")

await page.getByPlaceholder("get value from placeholder attribute")

await expect(page.getByRole('heading', { name: 'Sign up' })).toBeVisible();

await page.getByTestId("id of webelement")

await page.getByText("visible text")

await page.getByTitle("title of page")
```

—we can store locators in variables and use them

delay will take that much seconds between words.

use locator for xpath or css.

```
//we can store the locator and then use it also like selenium.
   test("login 4",async function({page}){
       await page.goto("https://ineuron-courses.vercel.app/")


      let loginbutton= await page.getByText("Log in")
      loginbutton.click();

      let newuserlink=await page.getByText("New user? Signup");
       newuserlink.click();

                                        //type for typing.
```

```
    await page.locator("//input[@id='name']").type("manoj", {delay:1000})
  })
```

—click

for clicking.

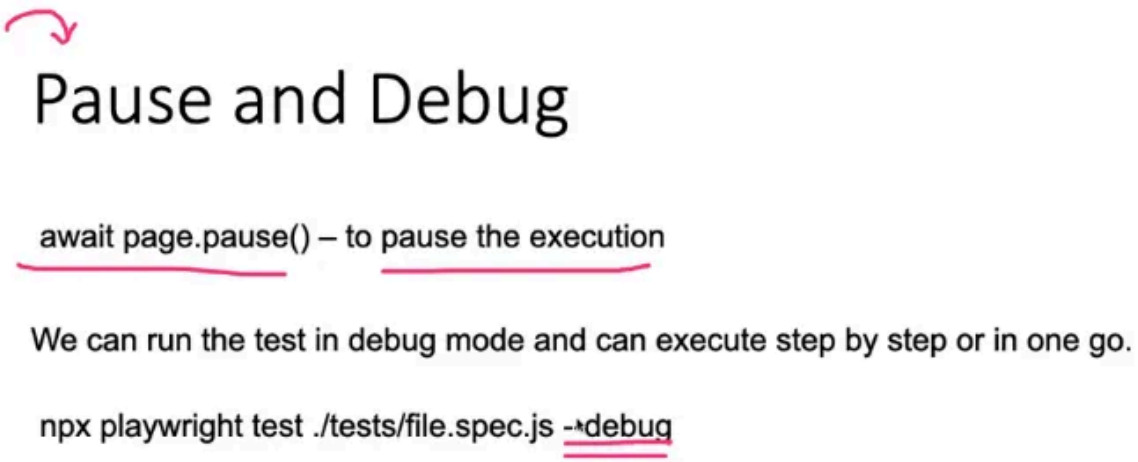
```
await page.getByLabel("Testing").click()
```

—we get strict mode violation when we get multiple web elements for same locator when running the code.

```
//dropdown. we can select by index, value.
    await page.locator("//select[@id='state']").selectOption("Assam")
```

—first method

```
//checkbox or radio button.
    //using first, click on the first testing if many are present.
    await page.getByLabel("Testing").first().click()
```

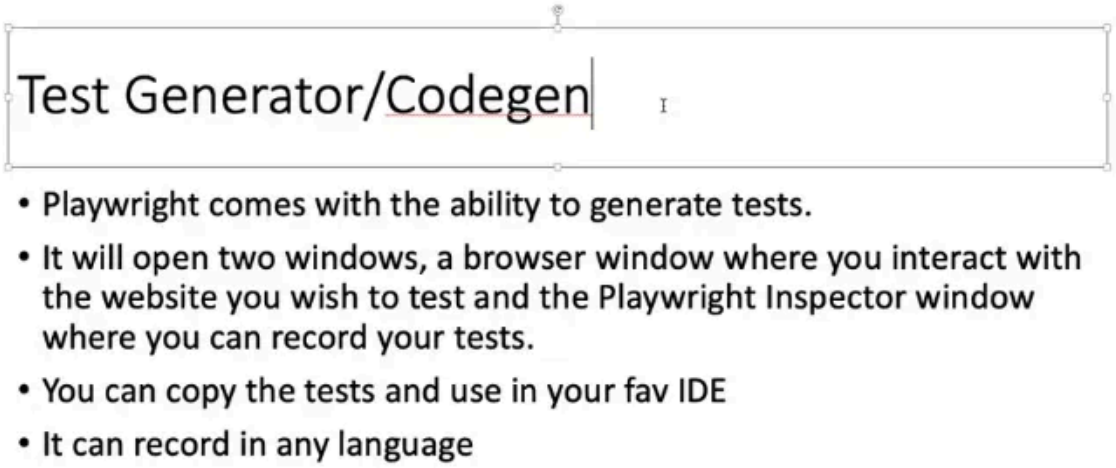



```
await page.pause();
```

this will open pw inspector and we can debug and go line by line.

—

auto code generator.



to know what all we can use inside codegen:

npx playwright codegen --help

to open url and record-

npx playwright codegen https://ineuron-courses.vercel.app/

simulates user actions on pw inspector.

can change language.